One User, Several Machines

Designing the SARSI Worker Layer

Server-Side Communicators Without Execution Authority, Five Work Agents, and a Vault That Only Ever Answers Yes or No

Chengshuai Yang
NextGen PlatformAI C Corp.
spiritai@platformai.org

August 2, 2026

Abstract

Two prior designs specify the console from opposite ends. *One Chat Box, Five Agents* specifies the agents a user meets; a session-level guide specifies the twenty-seven-node loop by which one `sarsi-claude` session is driven to a verified result. Between them sits a layer neither specifies: what stands between a person's request and a session on one of their machines.

This paper designs that layer. A user meets one chat box and chooses whom they are addressing — the manager, or the *communicator* of one work agent. Communicators run on the server and are always awake. Neither they nor the manager do any work: they plan, they answer, and they emit a typed directive to a *worker* running on one of the user's own machines. The worker is the only component that may drive `sarsi-claude`, and `sarsi-claude` is the only component that touches the task.

The central result is a separation argument. A component that is always available and may also execute is a remote-execution service on the user's hardware, available whenever the service chooses; and a component that may execute and must be always available is a machine the user cannot switch off. Availability and execution authority are therefore properties of *different* components, or one of two guarantees the user needs is lost (Proposition 1). The cost is one network hop per unit of work, and this paper pays it deliberately.

Two further results follow the same shape. A server may plan work that *requires* a secret without ever being able to learn it, because what crosses the network is a requirement and a boolean rather than a value (Proposition 2); the vault answering that boolean sits on the user's machine and decides in two stages, standing policy and then a per-use question, with promotion between them reserved to the owner. And an agent that both reads an untrusted channel and may act on its contents has converted that channel into a control interface for anyone who can write to it (Proposition 3) — which is why the work agent reads and drafts mail but may never send it, and why an instruction inside an email is evidence that someone asked, never authority to comply.

Five work agents are specified — `work`, `social`, `funding`, `jobs`, and the personal agent `abraham` — together with four workspace tiers, an append-only shared log whose contradictions are surfaced rather than resolved by clock order, and a phone that is a control surface and never a worker. Nothing described here is built.

Keywords: SARSI; multi-agent systems; personal agents; agent governance; capability separation; untrusted input; secret management; system design.

Contents

1 Introduction **4**
1.1 The layer that was missing . . . 4
1.2 The five planes . . . 4
1.3 Results . . . 4
1.4 Scope and status . . . 5

2 Availability Without Authority **5**
2.1 The argument . . . 5
2.2 What the rule buys, stated as a threat model . . . 5
2.3 Placement, and the refusal to queue . . . 6

3 The Vault **6**
3.1 A question, not a store lookup . . . 6
3.2 Two stages, and why promotion is reserved . . . 6
3.3 Where the secret's blast radius ends . . . 7

4 Outward Acts, and Untrusted Input **7**
4.1 Drafting is not sending . . . 7
4.2 Reading a channel the world can write to . . . 7

5 The Roster **8**
5.1 `work` — the salaried job . . . 8
5.2 `social` — one read a day, and influence . . . 8
5.3 `funding` — applications for the company or the research . . . 8
5.4 `jobs` — CV, and filling in the forms . . . 9
5.5 `abraham` — the personal agent . . . 9
5.5.1 Three differences in kind . . . 9
5.5.2 Five rules . . . 10
5.5.3 Two characteristic failures . . . 10
5.5.4 Deployment status . . . 11

6 Four Workspace Tiers **12**
6.1 Append-only, and contradictions that are surfaced . . . 12

7 Surfaces **12**
7.1 One chat box, and an explicit choice of counterpart . . . 12
7.2 The phone is a control surface . . . 13
7.3 Choosing the model . . . 13
7.4 Independence . . . 13

8 Failure Modes This Layer Adds **13**

9 Build Order **13**

10 What This Design Owes **14**

11 Conclusion **14**

A Invariant Checklist **15**

PART I — THE SEPARATION

1 Introduction

1.1 The layer that was missing

Two designs bracket this one. Above, *One Chat Box, Five Agents* [2] specifies what a user meets and how agents influence one another; its successor [1] specifies what changes once an account owns several machines. Below, the session-level guide specifies the loop that drives a single `sarsi-claude` session from a stated mission to a verified result — twenty-seven nodes, an independent verifier, permission gates, a stuck detector, and the owner's controls over all of it.

Between the two is a question neither answers: *when a person asks for something, what decides which machine does it, what prepares the task, what holds the secrets it needs, and what stops the result from reaching the outside world unreviewed?* That is the worker layer, and it is the subject of this paper.

The seam

This design meets the session design at exactly one node, called `ASG`: the worker creates a task and registers its goal. Everything below `ASG` is the session loop, already specified, and is not restated here. Everything above it is new.

1.2 The five planes

Table 1: Where each component runs, how many exist, and whether it may drive a session.

Plane	Runs on	Instances	May drive `sarsi-claude`?
Manager	server only	one per user	no
Communicator	server only	one per *agent name*	**no — see §2**
Worker	the user's machines	one per agent name *per machine*	**yes — only this**
`sarsi-claude`	the user's machines	one per task	it *is* the work
Vault	the user's machine	one per machine	no — it answers a question

1.3 Results

Three results

1. Availability and execution authority belong to different components (Proposition 1). Otherwise the user loses either the ability to reach an agent at any time or the ability to stop execution at any time. This is why a communicator may tell a worker to work and may never do the work.
2. A server can plan work requiring a secret without being able to learn it (Proposition 2). What crosses the network is a *requirement* and a *boolean*, never a value, so the blast radius of a compromised server excludes the user's credentials entirely.
3. Reading an untrusted channel and being able to act on it are jointly a control interface (Proposition 3). Mail, web pages and documents are inputs at the weakest evidence rung; an instruction found inside one is evidence that somebody asked, never authority to comply.

1.4 Scope and status

Nothing in this paper is built. It is a design under review, and the sections marked *owed* in §10 are gaps rather than omissions. The self-awareness substrate [4, 6], the fleet semantics [3, 5] and the session loop are assumed and not restated.

2 Availability Without Authority

2.1 The argument

Proposition 1 (Separation of availability from execution authority)**.** *Let a user require both:*

(a) ***reachability*** *— an agent that will accept a request at any time, including while every one of the user's machines is asleep; and*

(b) ***revocability*** *— the ability to stop all execution on their machines at any time, without losing (a).*

Then no single component may be both always available and capable of executing on the user's machines.

Argument by cases. Suppose one component is always available and may execute. To obtain (b) the user must stop that component's execution; but the component is the same one that provides (a), so stopping it forfeits reachability. The user is left choosing between being able to ask and being able to stop.

Suppose instead the executing component is required to be always available. Then (b) fails by definition: an always-available executor is precisely a machine that cannot be switched off.

Hence (a) and (b) are satisfiable only by distinct components: one always available and unable to execute, one able to execute and freely stoppable. □

Rule 1 (A communicator never operates `sarsi-claude`)**.** *The manager and the communicators may compose plans, converse with the user, choose a machine, and emit a typed directive. They may not drive a session, type into a terminal, read a file on a user machine, or execute anything there. The always-available components are deliberately the powerless ones.*

2.2 What the rule buys, stated as a threat model

The rule is not fastidiousness. Without it, the server is a service that may run code on the user's hardware at a time of its own choosing, and three consequences follow immediately, none of which requires anybody to behave badly.

1. **Compromise of the server becomes compromise of every machine.** With the rule, a compromised server can emit directives; workers still apply their own admission rules, the vault still refuses, and outward acts still stop at the owner.
2. **"Off" stops meaning off.** A user who closes their laptop has stopped execution. A user who wishes to stop execution while remaining reachable has, under the rule, exactly one action available and it is local.
3. **Latency becomes a correctness property rather than a performance one.** Because the server cannot act during a partition, there is no state in which it believes it is acting and is not — the failure that the multi-host design [1] spends a full section preventing at the level of controls.

The cost, stated plainly

Every unit of work costs a hop and a placement decision. A design in which the communicator simply did the work would be faster and simpler, and would forfeit both guarantees in Proposition 1. This paper

pays the hop.

2.3 Placement, and the refusal to queue

A directive must be placed on a machine. The communicator holds no machine's state — it is on the other side of a network from all of them — so placement is decided on *organizational* grounds: which machines carry this agent name, which report the required tool, which have reported recently, and which the user prefers.

Rule 2 (Unplaceable work is reported, not queued)**.** *If no machine carries the required tool, or none is reachable, the console says so and names what is missing. It does not hold the directive pending. An unplaceable directive that waits is indistinguishable, at the chat box, from work in progress.*

Rule 3 (Stale is unknown, not idle)**.** *A machine that has not reported within the reporting interval is* unknown*. It is not a candidate for placement, and it is never described to the user as available.*

3 The Vault

3.1 A question, not a store lookup

The user's secrets — mail credentials, card numbers, site accounts — live on their machine and are held by one component whose entire interface is a question.

Proposition 2 (Planning without disclosure)**.** *A server may compose a plan whose execution requires a secret s without s ever crossing the network, provided the plan names the* requirement *for s rather than s itself, and the requirement is resolved on the machine that holds s.*

Construction. Let the directive contain a requirement token r naming a class of secret ("mail credentials for account A"). The worker receiving the directive queries the local vault with $(r, \text{purpose})$. The vault returns ALLOW or DENY; on ALLOW it supplies s to the local worker only. The messages crossing the network are r (from server) and, at most, the boolean and the outcome (to server). Neither is a function of s's value, so no observer of the network learns s. □

What the vault may never do

Send a secret to the server, a communicator, or the manager • place a secret in a directive, a report, a workspace, a plan, or a prompt • allow any agent to enumerate or read the store — the only interface is the question • treat a per-use approval as a standing one.

3.2 Two stages, and why promotion is reserved

Rule 4 (Promotion is an owner act)**.** *A per-use approval never becomes a standing rule by repetition. Only the owner writes stage-1 policy.*

Without Rule 4 the obvious optimisation — *"you have allowed this five times; shall I stop asking?"* — becomes an agent acquiring standing authority by persistence, and the acquisition is invisible in exactly the case where it matters, since the fifth approval looks like the first four. The rule costs the user some repetition and is the reason the stage-1 policy language is the most urgent unbuilt item in §10: until it exists, everything falls to stage 2.

Table 2: The vault decides in two stages.

Stage	What it is	Answers alone when
1 · standing policy	rules the owner wrote once: *"`work` may read mail, never send"*	a rule matches — allow or deny, with no interruption
2 · per-use question	the owner is asked, for this use, naming the secret and the purpose	no rule matches, **or** the rule allows but the act is outward

3.3 Where the secret's blast radius ends

Proposition 2 has a corollary worth stating because it is the design's strongest security property and it is free: **the set of parties that can learn the user's secrets is the set of processes on the user's own machine.** The server is not in that set, and neither is any other of the user's machines — a vault is per machine, and secrets do not migrate any more than capability does [1].

4 Outward Acts, and Untrusted Input

4.1 Drafting is not sending

Rule 5 (Every outward act stops at the owner)**.** *An agent may compose anything. An act that leaves the machine and reaches a person — a sent email, a published post, a submitted application, a payment — requires an owner grant naming* that act*. A grant may cover a class and be bounded by uses; each use spends one; and no grant is ever inferred from the quality of the draft.*

This single rule is the distinguishing constraint of four of the five work agents, and stating it once prevents four weaker restatements. It also fixes the correct division of labour: composition is where the agent adds value and is unconstrained; transmission is where the consequences are, and is reserved.

Rule 6 (Approved and published must be identical)**.** *What the owner approved and what is transmitted are byte-identical. No reformatting, truncation, or template substitution occurs between approval and transmission.*

4.2 Reading a channel the world can write to

Proposition 3 (An acting reader is a control interface)**.** *Let an agent G read a channel C that arbitrary third parties may write to, and let G be able to perform actions A. If G may treat instructions found in C as directives, then any party able to write to C can invoke any action in A. The composition of "reads C" and "acts on what it reads" is a remote-invocation interface for A, whether or not one was intended.*

The channels this applies to are exactly the ones the work agents are for: a mailbox, a web page, a job-application form, a funding call document, a social feed. Anyone can send mail.

Rule 7 (An instruction inside a message is not an instruction to the agent)**.** *Content read from any channel is input at the weakest evidence rung. A message saying* "please wire the invoice" *is evidence that somebody asked. It is never authority to do it, and it never satisfies Rule 5.*

Rules 5 and 7 interlock, and the interlock is the point: even if an agent were persuaded by a hostile message, the act it was persuaded to perform is outward, and outward acts stop at the owner. The design does not rely on the agent not being fooled.

PART II — THE FIVE WORK AGENTS

5 The Roster

Table 3: Five work agents, one base worker, one vault. The last column is Rule 5 in each agent's own terms.

Name	For	Tools	Stops at the owner
`sarsi-worker`	the base worker; any task with no better home	shell, editor, browser	—
`work`	the salaried job	QuPath, MATLAB, whatever that job needs	**sending** mail
`social`	one daily read, and influence	browser	**posting**
`funding`	applications for the company or the research	browser, documents	**submitting**
`jobs`	CV, and filling in application sites	browser, documents	**submitting**
`abraham`	the personal agent: the user's own life	browser, calendar, documents	**everything outward**
`vault`	secrets, and the answer	none	it *is* the gate

5.1 `work` — the salaried job

Drives the tools that job requires, on the machine where they are installed, through `sarsi-claude`. Mail is its distinguishing feature and its sharpest constraint: it may read the mailbox, triage it, summarise it, say what needs the user, and draft replies in the user's voice. It may not send, and by Rule 7 it may not act on a request found inside a message.

Characteristic failure: a draft that is correct in tone and wrong in commitment — agreeing to a date, a scope, or a price the user would not have agreed to. Mitigation is that sending is reserved, which makes this failure visible rather than consequential.

5.2 `social` — one read a day, and influence

Two jobs that must not be confused. **Inbound:** gather, de-duplicate across sources, rank, and present a single digest, on the promise that reading it is *enough*. **Outbound:** draft for the user's channels; every post stops at the owner, under Rules 5 and 6.

Ranking is decision-conditioned rather than popularity-conditioned:

Rule 8 (An empty day is reported empty)**.** *The digest is never padded to length. A digest padded on quiet days teaches the reader to skim, and a digest that is skimmed has no value on the days it matters.*

5.3 `funding` — applications for the company or the research

Holds the state that is genuinely hard: which programme, which deadline, which documents exist, which are stale, what was submitted where and when. Its characteristic failure is a *plausible* application — well-formed, on time, and wrong about an eligibility fact — so it carries one additional rule: **every eligibility claim cites a source the owner can open.** Submission stops at the owner.

Table 4: Digest ranking signals, strongest first. Weights are a first guess and are owed measurement (§10).

Signal	Rung	Why it ranks there
Measured engagement — what the user opened, saved, replied to	L2	the only interest signal that is not self-reported
Declared interest	L5	authoritative for preference, and for nothing else
Consequence for a pending decision	—	a deadline outranks an interesting essay
Novelty after cross-source de-duplication	—	de-duplicate *before* ranking, or the fourth telling wins on volume
Source strength	—	a primary document outranks a summary of it
Decay	negative	yesterday's item earns its place again or leaves

5.4 `jobs` — CV, and filling in the forms

Writes and tailors the CV, then completes application sites over the web. It is asked, routinely, for facts that are the owner's rather than the agent's: salary expectation, availability, a reference's contact details. **It asks rather than infers.** An invented answer on a submitted form is the failure mode with the longest tail, because it is discovered by someone else, later, and cannot be retracted.

5.5 `abraham` — the personal agent

The user's life rather than their job: appointments, travel, purchases, household, family logistics, subscriptions, renewals. It has the *loosest scope* and the *tightest authority*, and conflating those would be the error. Tools: browser, calendar, documents, and — uniquely — payment instruments, always through the vault.

5.5.1 Three differences in kind

It is specified separately rather than as "another agent with different tasks" because three properties hold for it and for none of the other four.

Table 5: Why `abraham` needs its own rules.

Property	Why the others do not share it
Its acts land on people who never opted into an agent	the other four act on artefacts — figures, drafts, applications, a CV — that no third party is waiting on. A message to a relative arrives at a person who believes they are corresponding with the owner
Its acts include irreversible ones	a bad draft is embarrassing and retractable; a non-refundable booking is money spent. `work` and `social` can withdraw nearly anything they get wrong
Its subject matter is largely other people's personal data	the calendar holds other people's whereabouts; family logistics hold a child's schedule. None of those people agreed to anything

5.5.2 Five rules

Rule 9 (The approval states the cost of undoing)**.** *An owner approval for an irreversible or costly-to-reverse act shows the reversibility and its expiry —* "£840, free to cancel until Tuesday 09:00, then £180" *— not the price alone. An approval showing only the cost of acting has asked the owner about half the decision.*

The rule generalises: `funding` and `jobs` also perform irreversible submissions. `abraham` is where it stops being optional, because its irreversible acts are frequent, monetary, and routine rather than exceptional.

Rule 10 (Standing policy is scoped by amount and counterparty, never by agent)**.** *The stage-1 policy language must be unable to express* "`abraham` may use the card"*, and able to express* "up to £40 at the grocery, twice a week"*. For this agent the broad form is refused by the grammar rather than discouraged by guidance.*

The rule exists because of who writes the dangerous policy and why: an owner interrupted too often writes the broadest rule that stops the interruptions. A policy language that permits the broad form will be used to write it, and the vault's two-stage design (§3) will have been defeated by its own friction.

Rule 11 (Third-party personal data does not go server-side)**.** *W_{user} and W_{name} are held on the server.* `abraham` *shares* decisions *upward and not the personal facts those decisions concern:* "booked the Tuesday appointment" *may enter W_{name}; whose appointment it is, and for what, remains in W_{host}.*

This narrows Principle 1 for one agent. *Share intent, not instruments* assumed that intent is safe to share because it is about the owner; for `abraham` some intent is about somebody else, and the somebody else is not a party to this system at all. The general form of the rule is therefore: **a tier may hold facts about the owner and about hosts; it may not hold facts about third parties who have no relationship with the console.**

Rule 12 (All four reserved classes are ordinary traffic)**.** `money`*,* `consent`*,* `publishing` *and* `legal` *are the decision classes withheld at every ceiling tier.* `abraham` *touches all four in a normal week — paying, agreeing on someone's behalf, posting a family photograph, signing something. It may* prepare *all four and* complete *none.*

A consequence worth predicting rather than discovering: `abraham` will abstain more often than any other agent, and abstention is the action that costs nobody and leaves the autonomy denominator. Its autonomy rate is therefore not comparable with the other four's, and reporting them on one axis would make the most carefully bounded agent look like the least capable one.

Rule 13 (It gathers in licensed domains; it does not advise)**.** *Health, legal and financial material may be collected, organised, and put in front of the owner. Recommendations in those domains may not be made.* "Your renewal is on the 14th and here are the three documents" *is in scope;* "take the cheaper policy" *is not.*

5.5.3 Two characteristic failures

Plausible taste. The gift, the phrasing to a relative, the choice of restaurant. Unlike a wrong figure, this failure is invisible until it reaches a person, and that person cannot complain to the owner in time to stop it. Mitigation: for any act with a person on the other end, the owner approval is a *composition* step rather than a rubber stamp — the owner reads the words and edits them — which is why Rule 6 is load-bearing here specifically.

Quiet accumulation. Subscriptions, renewals and standing bookings continue to cost after everyone has forgotten them.

Rule 14 (A recurring obligation is its own act class)**.** *Approving a recurring commitment approves* one schedule*, not an open-ended one. Each is resurfaced on a cadence together with what it has cost to date. An agent that can create recurring charges and never mentions them again has been given a budget nobody agreed to.*

5.5.4 Deployment status

Not built — as with the other four. The two dependencies this specification adds are recorded in §10: a policy grammar that can refuse Rule 10's broad form, and a source of reversibility data for Rule 9.

PART III — SHARING, SURFACES, AND WHAT IS OWED

6 Four Workspace Tiers

Table 6: What is shared, with whom, and where it lives.

Tier	Visible to	Holds	Lives
W_{user}	every agent, every machine	who the user is, standing preferences, policies, the do-not list	server, read-mostly
W_{name}	every worker of *one agent name*, on every machine	mission, plan, decisions, what was done, the entities this agent works with	server, held by the communicator, **append-only**
W_{host}	one worker on one machine	tools present, paths, sessions, resources	that machine; never leaves
W_{secret}	nobody	credentials, cards, accounts	the vault; ALLOW/DENY only

Design principle 1 (Share intent, not instruments)**.** *A goal, a plan, a decision and a fact about the world mean the same thing on every machine, so they are shared. A tool inventory, a filesystem path, a context reading and a resource level are* about a host *and mean nothing off it, so they are not. Promoting the second kind is how a fleet convinces itself it can do something it cannot.*

A grant whose scope names a path is therefore host-bound and stays in W_{host}: the same string on another machine denotes a different resource, and sharing it would manufacture authority over a directory nobody examined [1].

6.1 Append-only, and contradictions that are surfaced

Two workers of one agent name may act simultaneously on different machines. The naive shared document has a write conflict; the resolution usually reached for is last-writer-wins.

Rule 15 (W_{name} is an append-only log)**.** *Reports are appended with the reporting machine's provenance; the shared view is a fold over the log. Nothing is overwritten, so simultaneous action cannot clobber.*

Rule 16 (Contradictions are surfaced, never resolved by clock)**.** *When the fold finds incompatible reports — one machine reports an application submitted, another reports it not submitted — both are presented with their machines and times, and the user is asked. Last-writer-wins would silently select the later* clock*, which is not the later* truth*, and a contradiction between two of the user's own machines is precisely the event a person should see.*

7 Surfaces

7.1 One chat box, and an explicit choice of counterpart

The user addresses either the manager or one communicator. Workers and sessions are not conversational counterparts: a worker is reachable through its communicator, a session through its worker. The choice is explicit rather than inferred because a router that guesses is wrong often enough that users begin writing prompts to steer the router instead of describing the work.

Every answer names **which machine** did the work and **at what authority** the claim stands — verified, recorded, or believed. In a fleet, "it worked" is an incomplete sentence.

7.2 The phone is a control surface

Rule 17 (No worker on the phone)**.** *A phone carries the chat box, approvals, vault questions, the digest, and the controls. It carries no worker, no session, and no tool use.*

The capability argument is obvious — a phone rarely has the tools and is usually asleep. The design argument is better: **approval is exactly the function a user wants available away from their desk, and execution is exactly the function they do not.** A phone that could run work would be an unsupervisable machine in the user's pocket, and it holds the most sensitive vault they own.

7.3 Choosing the model

Design principle 2 (The model is an engine, not an authority)**.** *No permission, ceiling, verdict or grant derives from which language model is running. Switching engines changes cost and quality; it changes nothing about what an agent may do.*

Two consequences. A verifier should not be the same instance that produced the work, and running it on a *different* engine is the cheapest available independence. And the engine is recorded with every outcome, since "it got worse last week" is answerable only if it is.

7.4 Independence

This console does not import from, depend on, or share a registry, store, process, or identity with the AI4Science agent line. Where a capability exists in both it is duplicated. The cost is real and it purchases the ability to change either without the other.

8 Failure Modes This Layer Adds

9 Build Order

Ordered so that each item is enforceable when it lands rather than correct in principle and unenforced.

1. **The typed directive, with the no-execute rule enforced.** Rule 1 is free to state and expensive to retrofit; the directive and its enforcement are written together.
2. **The vault, both stages, with a policy language.** Until stage 1 can be written, everything falls to stage 2 and the user is interrupted constantly — which is how a safety mechanism gets switched off.
3. **The outward-act gate.** Rule 5 before any agent can send, post or submit anything.
4. **Placement, with Rules 2 and 3.**
5. W_{name} **append-only, with the fold.**
6. **The five agents**, in the order `work`, `social`, `jobs`, `funding`, `abraham` — `abraham` last, because it is the least specified and has the widest scope.

Table 7: What goes wrong here, and what catches it.

Failure	What it looks like	What catches it
Silent queue	work waits on an unreachable machine and looks in progress	Rule 2
Placement on a ghost	a directive sent to a machine that stopped reporting	Rule 3
Mail as remote control	a message persuades an agent to act	Rules 7 + 5, jointly
Authority by persistence	repeated approvals become a standing grant	Rule 4
Secret in a report	a credential travels upward inside evidence	vault interface (§3)
Clock as truth	two machines disagree, the later one wins silently	Rule 16
Padded digest	a quiet day is filled to look busy	Rule 8
Reformat after approval	what was approved is not what was sent	Rule 6
Invented form field	a plausible answer is submitted and cannot be retracted	`jobs` asks rather than infers

10 What This Design Owes

1. The stage-1 policy language does not exist, and item 2 of the build order depends on it. Rule 10 now constrains it further: the grammar must be *unable* to express the broad form. A language that permits it has failed before it is written.
2. **Reversibility has no source.** Rule 9 requires the approval to state what undoing costs and when that changes. Nothing supplies it: it is not a property of the act, and scraping it from a vendor's page would present an inference as a fact. It is most likely declared per act class and marked *unknown* otherwise — and *unknown* must render as unknown, never as free.
3. The fold over W_{name} is unspecified: how the current view is computed, and how long the log is retained.
4. Rule 16 surfaces a contradiction and asks. What the user can *do* about it is undesigned.
5. The digest weights in Table 4 are a first guess. The signals are argued; the weights are not measured.
6. **Nothing here is built.**

11 Conclusion

The worker layer is a single separation applied three times. Availability is separated from execution authority, so that a user may always ask and may always stop. Knowledge of a secret is separated from the ability to plan around it, so that a server may compose work it could never itself perform. And composition is separated from transmission, so that an agent may write anything and send nothing.

Each separation costs something concrete — a network hop, a local component, an approval — and each buys a guarantee that no amount of care in the agents themselves could provide. That is the test this design applies to itself: a property that depends on an agent behaving well is not a property of the system, and the three propositions here were chosen because they survive an agent that does not.

A Invariant Checklist

W1. No server-side component executes on a user machine.
W2. Only a worker creates a session or assigns it a task.
W3. No secret is present in any directive, report, workspace, plan, or prompt.
W4. The vault's only interface is a question; no agent enumerates or reads the store.
W5. A per-use approval never becomes standing policy without an owner act.
W6. Every outward act is covered by a grant naming that act; each use spends one.
W7. What was approved and what was transmitted are byte-identical.
W8. Content read from any channel is treated as input, never as a directive.
W9. Unplaceable work is reported; it is never held pending.
W10. A machine that has not reported within the interval is unknown, not idle.
W11. W_{name} is append-only; contradictions are surfaced, not resolved by timestamp.
W12. Host-scoped facts and path-scoped grants never enter W_{name} or W_{user}.
W13. No worker or session runs on a phone.
W14. No permission, ceiling, verdict or grant depends on which model is running.
W15. Every answer names the machine that produced the result and the authority of the claim.
W16. An approval for an irreversible act states the cost of undoing it and when that cost changes; unknown reversibility renders as unknown, never as free.
W17. No standing policy grants an agent a payment instrument without naming both an amount limit and a counterparty class.
W18. No tier above W_{host} holds personal data about a third party who has no relationship with the console.
W19. Approving a recurring obligation approves one schedule, and each is resurfaced with its cost to date.

References

[1] Chengshuai Yang. One account, many machines: Designing the multi-host SARSI console — per-host agents, host-indexed evidence, channels that can drop, and what a machine takes with it when it leaves. Preprint. The immediate predecessor of this paper, 2026.

[2] Chengshuai Yang. One chat box, five agents: Designing the manager-fronted SARSI console — per-agent self-awareness, nested workspaces, and the four channels by which agents affect each other. Preprint, 2026.

[3] Chengshuai Yang. Functional self-awareness for hierarchical SARSI multi-agent systems: Version 3.0 — a brain-inspired manager–specialist architecture with bidirectional influence, nested workspaces, and safe user interruptibility. Preprint. Referred to as "Version 3.0", 2026.

[4] Chengshuai Yang. Functional self-awareness for SARSI agents: Version 2.0 (history-aware revision) — selective episodic encoding, replay, consolidation, and bounded retrieval. Preprint, 2026.

[5] Chengshuai Yang. A manager is not a controller: Global-workspace coordination for SARSI fleets. Companion paper on coordination semantics, 2026.

[6] Chengshuai Yang. Functional self-awareness for SARSI agents: Version 2.0 (corrected and expanded). Preprint. The single-agent architecture, 2026.